Übung 3 · Ein Agent mit Werkzeugen

Cloud-native AI · WS 2026/27 · Praktikumsabgabe P3

- **Start:** Übung am Dienstag, 03.11.2026, 11:30, O/201
- **Abgabe:** nach VL 6, die Termine für Termin A und Termin B stehen in der Moodle-Aufgabe
- **Arbeitsform:** einzeln oder zu zweit, wie in der Moodle-Gruppeneinteilung eingetragen
- **Aufwand:** etwa 4 Stunden für Teil 1 bis 3, Teil 4 und 5 folgen mit VL 6

Worum es geht

Bisher hat unser Code entschieden, was passiert: welche Datei gesucht wird, welcher Prompt läuft, was in den Kontext kommt. Ab heute entscheidet das Modell selbst. Es bekommt Werkzeuge, wählt in einer Schleife, welches es mit welchen Argumenten aufruft, liest das Ergebnis und macht weiter, bis es antworten kann. Das ist ein Agent.

Ihr baut die Schleife ohne Framework, schreibt eigene Werkzeuge, betreibt einen Teil davon als eigenen Service über das Model Context Protocol (MCP) und messt, wo der Agent scheitert. Grundlage ist das Skript zu **VL 5** (Agenten und Werkzeuge).

Teil	Inhalt	Aufwand	
1	Die Agent-Schleife	ca. 1,5 h	Kern
2	Eigene Werkzeuge, lokal und als MCP-Server	ca. 1,5 h	Kern
3	Verbessern und messen	ca. 1 h	Kern
4, 5	Orchestrierung und Memory, README	folgen mit VL 6	Kern
Z	Zusatz: ReAct ohne Function Calling	ca. 2 h	Zusatz

Einrichten (am besten zu Hause)

Entpackt **„Übung 3 · Starter (ZIP)“** als neuen Ordner `uebung_3/` in euer Repository. Der Starter nutzt das Volume `cnai-ollama-modelle` aus Übung 2. Wer es nicht hat, legt es einmal an: `docker volume create cnai-ollama-modelle`.

```
cd uebung_3
cp .env.example .env       # oder eure .env aus Übung 2 übernehmen und ergänzen
docker compose run --rm --build app werkzeuge.py
docker compose run --rm app mcp_werkzeuge.py
```

Der erste Aufruf baut zwei Images und lädt `qwen2.5:3b` (ca. 1,9 GB) als zweites Modell. Der zweite zeigt die Werkzeuge des MCP-Servers, genau so, wie das Modell sie sieht. Übernehmt ihr eure `.env` aus Übung 2, ergänzt `MCP_URL` und `HEUTE` aus `.env.example`.

Was ist neu im Starter?

Datei	Was sie tut
`app/agent.py`	der Agent: Schleife, Werkzeugausführung, Ausgabe
`app/werkzeuge.py`	lokale Werkzeuge: `suche_wissen`, `lies_datei`, `heute`
`app/mcp_werkzeuge.py`	holt die Werkzeuge vom MCP-Server und macht sie aufrufbar
`app/workflow.py`	zum Vergleich: ein fester Routing-Workflow ohne Schleife
`app/messen.py`, `app/aufgaben.json`	13 Testaufgaben und die Messung nach Huyens Fehlermodi
`campus_mcp/server.py`	der MCP-Server: Raumbelegung und Mensa, eigener Container
`app/wissen/`	die Wissensbasis aus Übung 2

`HEUTE=2026-11-03` in der `.env` hält die Uhr an. Damit sind eure Messungen reproduzierbar und vergleichbar, egal an welchem Tag ihr messt.

Teil 1 · Die Agent-Schleife (Kern)

Lest zuerst `werkzeuge.py` und den oberen Teil von `agent.py`. `werkzeug_ausfuehren()` ist fertig: Es prüft einen Werkzeugaufruf des Modells, führt ihn aus und gibt Fehler als Text zurück statt eine Exception zu werfen.

Aufgaben:

1. Setzt **TODO 1** in `agent.py` um, die Schleife aus dem Skript. Testet sie: `bash docker compose run --rm app agent.py "Was gibt es heute in der Mensa?" docker compose run --rm app agent.py "Ich bin am Tag meiner Klausur krank. Was muss ich tun?" docker compose run --rm app agent.py "Ist O-108 am Donnerstag um 14 Uhr frei?"` Jeder Werkzeugaufruf erscheint mit `→`, jedes Ergebnis mit `←`. Beschreibt im README für jede der drei Fragen die Trajektorie: Welche Werkzeuge mit welchen Argumenten, und stimmt die Antwort?
2. Ordnet jeden Fehler, den ihr seht, einem Fehlermodus nach Huyen zu (Skript, Abschnitt „Wie Agenten scheitern“).
3. Lasst die erste Messung laufen und übernehmt die Tabelle ins README: `bash docker compose run --rm app messen.py` Ein Lauf über 13 Aufgaben dauert mit `llama3.2:3b` auf einem Laptop etwa drei Minuten. Die Buchungsaufgabe scheitert noch, das Werkzeug fehlt. Das ist gewollt.
4. Schaut euch bei einer Aufgabe mit mehreren Schritten die Eingabe-Tokens an. Warum steigen sie mit jedem Modellaufruf, obwohl die Frage gleich bleibt?
5. Setzt `--max-schritte 1` und stellt eine Frage, für die der Agent zwei Werkzeuge braucht. Was passiert, und warum ist ein Schrittlimit trotzdem unverzichtbar?

Teil 2 · Eigene Werkzeuge, lokal und als MCP-Server (Kern)

2.1 · Ein lokales Werkzeug

Sprachmodelle rechnen schlecht mit Datumsangaben. Setzt **TODO 2** in `werkzeuge.py` um: ein Werkzeug `tage_bis(datum)`, das die Tage von heute bis zu einem Datum ausrechnet. Tragt es mit Beschreibung und JSON-Schema in `LOKALE_WERKZEUGE` ein.

```
docker compose run --rm app agent.py "Wie viele Tage habe ich noch, bis die Prüfungsanmeldung endet?"
```

Für die richtige Antwort braucht der Agent drei Dinge: das heutige Datum, die Frist aus der Wissensbasis und die Rechnung. Beschreibt, ob und in welcher Reihenfolge er sie sich holt.

2.2 · Ein Werkzeug im MCP-Server

Der Server `campus_mcp/server.py` läuft als eigener Container. Bisher kann er nur lesen. Setzt **TODO 3a** um: ein Werkzeug `raum_buchen(raum, datum, uhrzeit, name)`. Der Docstring wird zur Beschreibung, die das Modell sieht. Nach jeder Änderung am Server:

```
docker compose restart campus-mcp
docker compose run --rm app mcp_werkzeuge.py
```

2.3 · Bestätigung für Schreibaktionen

`raum_buchen` verändert die Welt. Setzt **TODO 3b** in `agent.py` um: Schreibende Werkzeuge laufen nur nach einer Bestätigung im Terminal.

```
docker compose run --rm app agent.py "Buche mir O-108 am Freitag um 9:45 Uhr für die Lerngruppe Kim."
```

Aufgaben:

1. Testet einmal mit `j` und einmal mit `n`. Was macht das Modell nach der Ablehnung?
2. Woran erkennt `mcp_werkzeuge.py`, dass ein Werkzeug schreibend ist? Was ist an dieser Lösung schwach, und wo sollte die Information eigentlich herkommen? Tipp: MCP kennt dafür *Tool Annotations*.
3. Optional: Öffnet den **MCP Inspector** auf eurem Rechner (`npx @modelcontextprotocol/inspector`, Transport *Streamable HTTP*, URL `http://localhost:8000/mcp`) und ruft eure Werkzeuge ohne Modell auf. Wer Claude Desktop oder Cursor nutzt, kann den Server dort ebenfalls eintragen.

Teil 3 · Verbessern und messen (Kern)

Kleine Modelle scheitern vor allem an Datumsangaben und an zu kurzer Suche. Es gibt drei Hebel. Probiert **mindestens zwei** davon aus und messt jeden einzeln mit `messen.py` gegen den Stand aus Teil 2:

- **Prompt:** Schreibt `prompts/agent.v2.yaml` mit klaren Regeln, wann welches Werkzeug dran ist, und ein, zwei Beispielabläufen. `agent.v1.yaml` bleibt unverändert.
- **Werkzeug:** Macht `raum_frei`, `raum_buchen` und `mensa_speiseplan` fehlertolerant (Poka-Yoke): `datum` darf auch `heute`, `morgen` oder ein Wochentag sein, eine angehängte Uhrzeit wird ignoriert. Passt den Docstring an.
- **Workflow statt Agent:** Messt den festen Routing-Workflow aus `workflow.py` gegen euren Agenten.

```
docker compose run --rm app messen.py prompts/agent.v1.yaml prompts/agent.v2.yaml workflow
docker compose run --rm -e LLM_MODEL=qwen2.5:3b app messen.py prompts/agent.v1.yaml prompts/agent.v2.yaml workflow
```

Aufgaben:

1. Übernehmt die Tabellen ins README, für beide Modelle. Welcher Hebel hat bei welchem Aufgabentyp am meisten gebracht?
2. Bei welchen Aufgaben gewinnt der Workflow, bei welchen der Agent? Erklärt es mit dem Unterschied zwischen festen Pfaden und Schleife.

3. Welches Modell würdet ihr für diesen Agenten nehmen, und welches für den Workflow? Begründet mit euren Zahlen.
4. Wer Zugang zur Academic Cloud hat: Messt `qwen3-30b-a3b-instruct-2507` mit `agent.v1`. Was ändert ein großes Modell an den Fehlermodi?

Teil 4 und 5 · folgen mit VL 6

In VL 6 kommen Orchestrierung mit einem Framework, Memory und ein kleines Multi-Agent-Setup dazu, außerdem die README-Anforderungen für die Abgabe. Das Blatt wird dann ergänzt.

Zusatz · ReAct ohne Function Calling

Dieser Teil **entfällt**, wenn ihr am Semesterende mindestens 80 % Anwesenheit nachweist. Wer nicht sicher ist, gibt ihn mit ab. Zusatzteile sind nicht nachreichbar.

Das ursprüngliche ReAct-Paper (Yao et al., 2022) kam ohne natives Function Calling aus. Das Modell schrieb `Thought:`, `Action:` und wartete auf eine `Observation:`, die das Programm einfügte.

1. Schreibt `react.py`: eine Agent-Schleife, die dem Modell die Werkzeuge im System-Prompt beschreibt und eine Antwort im Format `Thought: … Action: werkzeug({"param": "wert"})` oder `Final Answer: …` verlangt. Euer Code liest die Aktion mit einem regulären Ausdruck aus, führt sie aus und hängt `Observation: …` an.
2. Messt `react.py` mit denselben Aufgaben gegen `agent.py`. Ergänzt dazu in `messen.py` eine Variante `react`.
3. Was ist robuster, was ist billiger, und bei welchem Modell? Beschreibt zwei Fehler, die nur bei der Textvariante auftreten.

Was ihr abgebt

In Moodle, Aufgabe „Übung 3 (P3)“: den **Link zu eurem Repository** und den Commit-Hash eures Abgabestands. Das Repository muss für mich lesbar sein: öffentlich, oder ihr ladet meinen GitHub-Account `PascalLaube` als Collaborator ein.

Harte Kriterien für Teil 1 bis 3, ergänzt mit VL 6:

☐ Das System startet mit den im README beschriebenen Befehlen nach einer beschriebenen `.env`.
☐ Keine Secrets im Repository, `.env` ist nicht committet.
☐ Abhängigkeiten sind gepinnt.
☐ TODO 1, 2, 3a und 3b sind umgesetzt, `messen.py` läuft durch.
☐ `raum_buchen` läuft im MCP-Server, nicht im Code des Agenten.
☐ Das README enthält die Trajektorien aus Teil 1 und die Messtabellen aus Teil 3.

Tipps

- **Erst einzeln, dann messen.** Testet jede Änderung mit einer Frage in `agent.py`, bevor ihr `messen.py` startet.

- **Schwankungen:** Auch mit Temperature 0 kann ein Lauf um eine Aufgabe abweichen. Wiederholt auffällige Messungen und notiert die Schwankung, statt sie wegzulassen.
- **Werkzeugaufruf als Text:** `llama3.2:3b` schreibt manchmal `{"name": "…", "parameters": …}` als normale Antwort, statt ein Werkzeug aufzurufen. Das ist ein echter Fehlermodus. Notiert ihn.
- **Buchungen sind flüchtig.** Sie liegen nur im Arbeitsspeicher des Servers. `docker compose restart campus-mcp` setzt alles zurück.
- **Der Server antwortet nicht?** `docker compose logs campus-mcp` zeigt Fehler beim Start, etwa einen Syntaxfehler in eurem neuen Werkzeug.
- **KI-Werkzeuge** dürft ihr nutzen. Ihr müsst jede Zeile eures Codes und jede Werkzeugbeschreibung erklären können.